

基于 RK3588 的多模态分析与自主搜救水上救生系统

摘要

本项目设计并实现了一套基于 RK3588 的多模态分析与自主搜救水上救生系统，针对水域溺水事故发生不及时、救援响应滞后、传统设备环境适应性差等痛点，构建了集“智能感知、溺水判定、主动施救、云端管控”于一体的全闭环救援体系。

系统以瑞芯微 RK3588 芯片为核心算力平台，依托其 6TOPSAI 算力与 NPU 多核异构架构，集成高清 130° 广角摄像头、麦克风阵列、压力传感器等多源感知设备，通过经 INT8 量化优化的 YOLOv8 视觉检测模型与 DNN 音频异常识别模型，实现复杂水域环境下溺水行为与呼救信号的实时融合检测。

当系统连续识别到溺水目标或检测到有效呼救声时，立即触发报警并上传云端。判定溺水风险后，通过 MQTT 协议向控制节点下发指令，驱动舵机自动释放智能救生圈。救生圈搭载视觉定位自主导航模块，在完成投放后能够自动锁定并持续跟踪溺水目标，通过双 PID 运动控制算法实时调整航向和推进速度，实现自主接近目标。落水人员抓靠救生圈后，压力传感器实时检测受力状态并反馈至网页端，形成“险情发现—自动报警—救生圈投放—自主追踪—救援反馈”的完整闭环。

系统同步开发 Vue3Web 管理端与 Uni-App 移动端，实现实时视频监控、报警推送、设备管理及历史记录查询等功能。经实际测试，系统识别准确率达到 92.1%，报警响应时间低于 500ms，具备良好的实时性、可靠性和工程应用价值，可广泛应用于河道、湖泊、水库、景区及校园等水域安全场景。

关键词：智能救援；边缘计算；RK3588；多模态感知；自主导航

第一部分 作品概述

1.1 功能与特性

该设备搭载瑞芯微 RK3588 八核芯片，集成 6TOPS 算力 NPU，可在复杂水面环境下完成 200ms 级实时数据处理，支撑 30FPS 高清视频流 AI 推理，为智能识别提供高性能边缘算力支撑。设备融合光电视觉与声学感知能力，通过 YOLOv8 双模型完成“人体目标检测—溺水状态判别”全链路判断，搭配音频模型检测呼救信号，结合防误判分析机制抑制误报，可在复杂光照、水波干扰的场景下实现稳定可靠的险情检测。

在救援执行与管控方面，系统基于视觉定位与双 PID 闭环控制算法，实现全局路径规划、动态避障与精准趋近，配合双无刷电机推进系统与流体型低阻流线结构，最高航速达 1.5m/s，可自动完成从识别到施救的全自主救援流程；同时通过 WIFI6/4G/5G 通信模块实现设备上云，配套 Web 管理平台与移动端小程序，支持实时视频回传、险情报警推送、设备状态监控与救援数据统计，实现水域全域可视化远程管理。

1.2 应用领域

本系统可适配多类水域安全场景，针对性解决传统救援模式的痛点：

（1）开放式公共水域救援

适用于海滨浴场、景区湖泊、水库等人员密集、监管范围大的场景。系统通过广角摄像头实现大范围无死角监控，AI 算法 24 小时不间断值守，突破人工巡逻的视野盲区与时效限制，尤其适配暑期客流高峰的应急响应需求。野外救援如图 1-1 所示。



图 1-1 野外救援示意图

(2) 泳池及水上乐园智能监管

针对室内外泳池、水上乐园场景，可辅助救生员完成常态化监测。优化后的行为识别算法可精准区分正常戏水与溺水挣扎，降低夜间、人流密集时段的监管压力，减少人力投入与运营风险。室内救援如图 1-2 所示。



图 1-2 室内救援示意图

(3) 河道码头应急响应

适配水流湍急、漂浮物多的复杂河道与码头环境。流体型低阻结构配合动态避障算法，可在急流、障碍物环境下稳定航行，为落水人员争取黄金救援时间，弥补传统救援设备在复杂水文环境下的机动能力不足。

1.3 主要技术特点

本系统基于“边缘智能、多维感知、自主控制”的设计理念，在工程实现上具有以下技术特点：

（1）异构边缘计算与硬件高集成设计

系统采用软硬件协同设计方法，以高算力低功耗的 RK3588 为核心总成，通过电路集成手段整合了 6TOPS 的异构 NPU 算力资源、高灵敏度声学采集矩阵以及 WiFi 6/4G/5G 多模态通信链路。硬件层具备宽温（-10℃~80℃）自适应能力，为前端高密度数据的毫秒级级联处理提供了高可靠的边缘硬件底座。

（2）串级级联视觉与音频双重校验机制

算法架构摒弃了单一的目标检测模式，采用“串级级联”的判定逻辑。前级由 YOLOv8 轻量化剪枝模型进行全时域人体目标检索，后级触发溺水状态细粒度行为判别。同时，视觉流与声学呼救信号在时域上进行交叉校验，引入“5s 内触发大于 2 次”的窗长滤波算法。通过这种视听双源融合的校验机制，工程上解决了复杂水面光照突变和波浪反光导致的算法失效问题。

（3）动态控制律解耦与流体动力学协同

系统实现了流体外形与控制算法的深度耦合。物理外形基于 Fluent 软件进行流场寻优，采用高浮力、低阻力的流体型拓扑结构；控制层将双无刷电机的非线性动力学模型进行解耦，内嵌双 PID 闭环控制律。算法根据视觉反馈定位自动完成全局路径规划与动态避障，在执行层面保证了转弯、趋近等动作的精准度，使设备具备 3 级海况下的自愈抗浪能力。

（4）分布式云端总线与可视化遥测网络

系统构建了端云协同的分布式网络架构。底层采用轻量级 MQTT 协议作为高频状态遥测与险情报警（<1s）的传输总线，上层基于 WebSocket 协议打通全双工高清视频流传输，端云交互延迟控制在 800ms 以内。通过业务逻辑下沉与标准化接口设计，实现了 Web 端与移动端小程序对多达 20 台网络终端的数字化全域可视化管控，并支持数据回流与在线 OTA 模型演进。

1.4 主要性能指标

主要性能指标如表 1-1 所示。

表 1-1 主要性能指标汇总

技术模块	性能指标
边缘计算	处理延迟 $\leq 200\text{ms}$; 工作温度 $-10^{\circ}\text{C}\sim 80^{\circ}\text{C}$ 。
AI 视觉识别	识别准确率 92.1%，误报率 $< 2\%$; 推理速度 30FPS。
音频识别	声识别率 $\geq 94\%$; 防误判 5s 触发大于 2 次。
自主导航	路径规划时间 $< 1\text{s}$; 定位误差 $< 0.5\text{m}$ 。
动力系统	最高航速 1.5m/s ; 推力 80N，抗浪能力 3 级海况。
通信管理	端云延迟 $< 800\text{ms}$; 最大组网设备数 20 台; 报警响应时间 $< 1\text{s}$ 通信管理。

1.5 主要创新点

（1）高性能边缘算力支撑

搭载瑞芯微 RK3588 八核芯片，集成 6TOPS 算力 NPU，可在复杂水面环境下完成 200ms 级实时数据处理，支撑 30FPS 高清视频流 AI 推理，保障险情识别的实时性。

（2）多模态智能感知识别

融合光电视觉与声学感知，通过 YOLOv8 双模型完成“人体目标检测 — 溺水状态判别”，搭配音频模型检测呼救信号，结合防误判分析抑制误报，实现复杂光照、水波干扰下的稳定检测。

（3）全自主导航救援执行

基于视觉定位与双 PID 闭环控制算法，实现全局路径规划、动态避障与精准趋近，配合双无刷电机推进系统与流体型低阻结构，最高航速 1.5m/s ，可自动完

成从识别到施救的全流程。

(4) 端云协同远程管控

通过 WIFI6 模块实现设备上云，配套 Web 管理平台与移动端程序，支持实时视频回传、险情报警推送、设备状态监控与救援数据统计，实现水域全域可视化管理。

1.6 设计流程

本项目从水域救援实际需求出发，先通过调研溺水事故数据明确“黄金四分钟”核心性能指标（响应时延 $\leq 200\text{ms}$ 、航行速度 $\geq 1\text{m/s}$ ），同时梳理浪高、人流、复杂光照等复杂水域场景的技术挑战；随后采用多学科协同设计模式推进研发，结构组完成流体型结构设计并通过仿真优化，硬件组实现 RK3588 主控芯片与 WiFi 6/4G/5G 通信模块的电路集成，算法组同步开展 YOLOv8 模型剪枝量化与避障算法开发；设计阶段同步开展多维度仿真验证，通过 Fluent 流体阻力仿真优化流体型结构曲率，依托 Gazebo 水域导航模拟验证 90°转弯的可行性，经 TensorRT 部署测试确认 30FPS 的实时识别速度；在此基础上推进三轮原型迭代，依次完成基础推进与识别功能验证、新增抗倾覆控制功能、集成 WIFI6 通信与多端调度能力；原型定型后开展全场景实景测试，分别在静水泳池测量航速与识别准确率、在开放水域完成 30cm 浪高抗浪测试、在夜间场景验证红外补光下的误报率水平；产品落地部署后，持续基于实际救援数据优化路径规划参数，并通过 OTA 远程更新算法模型，实现设备性能的持续迭代优化。

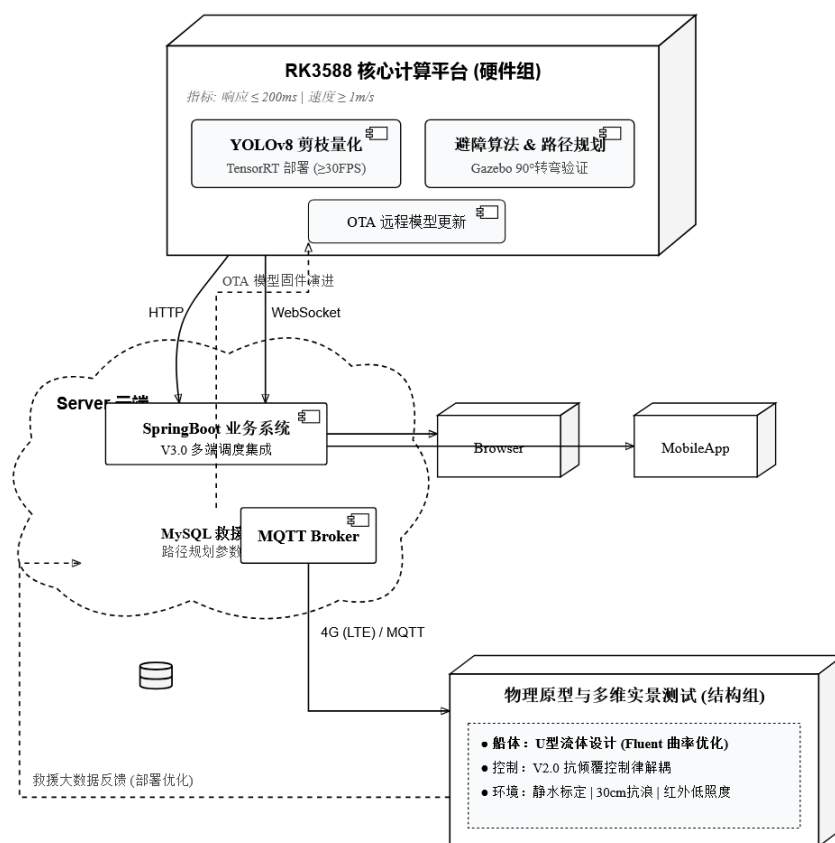


图 1-3 系统设计流程图

第二部分 系统组成及功能说明

2.1 整体介绍

2.1.1 系统总体架构

系统采用分层架构设计，整体分为感知执行层、边缘计算层、云端服务层和应用管理层四个部分，整体技术路线如图 2-1 所示。

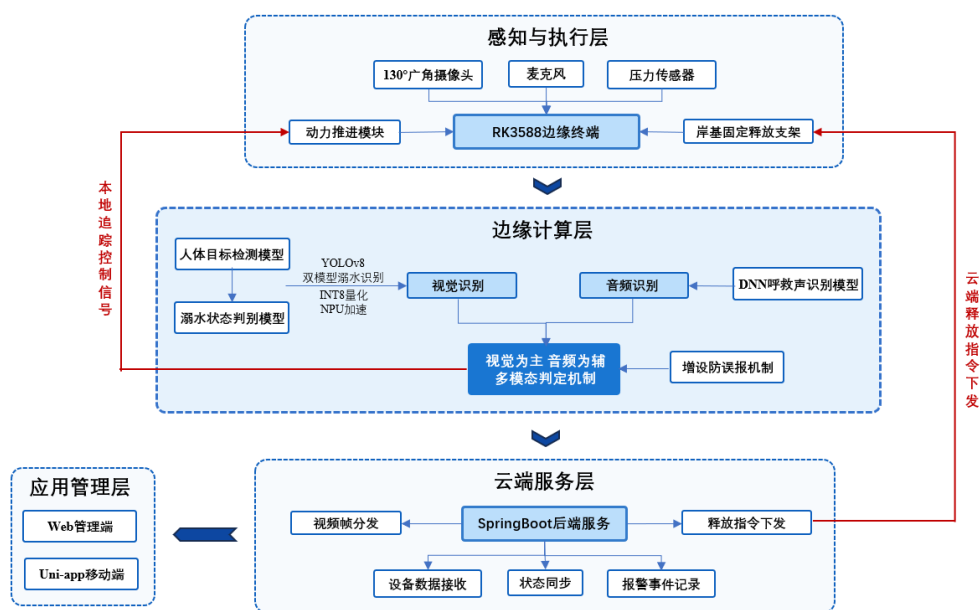


图 2-1 系统技术路线总览示意图

感知执行层部署于水域现场，主要包括 130° 广角摄像头、麦克风、压力传感器、岸基固定释放支架、声光报警组件和动力推进模块。摄像头用于采集水域画面，麦克风用于采集环境声音，压力传感器用于检测落水人员是否抓靠救生圈，岸基支架则负责救生圈固定、释放和现场警示。

边缘计算层以 RK3588 开发套件为核心，负责水域画面识别、呼救声辅助检测、检测结果解析和本地初步判断。视觉部分采用 YOLOv8 双模型完成人体目标检测与溺水状态判别，音频部分用于辅助判断是否存在有效呼救声。边缘端在完成识别后，将报警信息、目标坐标、设备状态和视频帧上传至云端。

云端服务层负责设备数据接收、报警事件记录、状态同步、视频帧分发和释放指令下发。当云端接收到边缘端上传的报警信息后，结合设备当前状态进行联动处理，并通过 MQTT 向岸基固定释放支架发送释放指令，同时将报警事件推送至 Web 管理端和移动端。

应用管理层包括 Web 管理端和移动端应用。Web 端用于实时视频查看、设备状态监控、报警管理和历史记录查询；移动端用于报警接收、设备状态查看和语音提示。通过多端协同，管理人员能够及时掌握水域现场情况和设备运行状态。系统总体架构图如图 2-2 所示。



图 2-2 智能救生圈技术架构示意图

2.1.2 功能模块与数据流设计

系统围绕“感知—识别—报警—释放—追踪—反馈”的救援流程进行功能模块划分，各模块之间通过边缘端、云端和应用端的数据交互完成协同工作，整体数据流如图 2-3 所示。

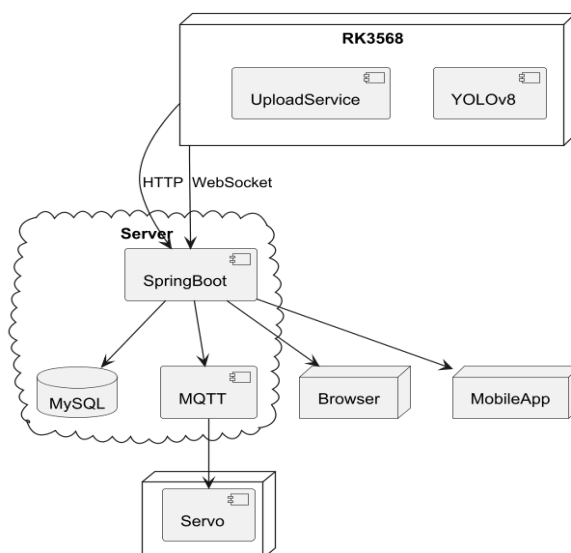


图 2-3 系统数据流图

(1) 水域感知模块。该模块由 130° 广角摄像头、麦克风和压力传感器组成。摄像头负责采集水域画面，麦克风负责采集现场环境声音，压力传感器用于

检测落水人员是否抓靠救生圈。感知数据首先进入边缘计算终端，为后续目标识别、呼救声辅助判断和救援状态反馈提供数据基础。

(2) 边缘识别与防误报模块。边缘计算终端对摄像头画面进行预处理和模型推理，先检测水面人员目标，再判断目标是否存在溺水风险，并输出目标数量、坐标位置、置信度等结构化结果。系统以视觉识别结果作为主要判断依据，结合呼救声识别结果、压力传感器状态和连续帧确认机制进行综合判断，减少水面反光、短暂遮挡、单帧误检或偶发噪声导致的误报警。多模态感知流程如图 3-2 所示。

(3) 报警上传与云端联动模块。当边缘端检测到疑似溺水风险后，将报警信息、目标坐标、设备状态等数据上传至云服务器。云端接收后记录报警事件，并将报警状态同步推送至 Web 管理端和移动端。当报警满足释放条件时，云端通过 MQTT 向岸基固定释放支架下发释放指令，执行端驱动舵机解除固定并释放智能救生圈，同时联动声光报警组件进行现场警示。

(4) 自主追踪与运动控制模块。救生圈入水后，根据视觉定位结果获取落水人员在画面中的位置，并结合双 PID 控制算法调整航向和推进速度，使救生圈能够自主靠近目标。在接近落水人员过程中，系统逐步降低速度，减少碰撞风险。

PID 控制流程如图 2-4 所示。

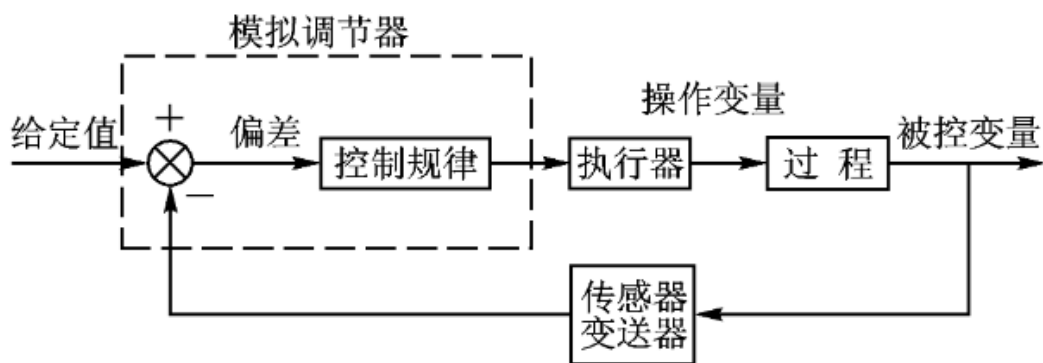


图 2-4 PID 算法流程图

(5) 视频推流与状态同步模块。边缘端将识别后的 JPEG 图像帧通过 WebSocket 上传至云端，云端接收后分发至 Web 管理端和移动端，用于实时视频显示。同时，设备状态、报警事件、温度信息和救援完成状态通过消息推送方式

同步至 Web 端和移动端，便于管理人员远程查看现场情况。

（6）救援反馈与历史记录模块。落水人员抓靠救生圈后，压力传感器状态上传至云端，云端更新救援完成状态，并向 Web 管理端和移动端同步反馈结果。系统同时保存设备信息、报警记录、释放状态和救援结果，便于后续查询和管理复盘。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

2.2.1.1 主控平台

传统的 51 单片机 I/O 少，ROM 空间不足，所以我们采用瑞芯微 RK3588 单片机作为主控器。瑞芯微 RK3588 运行速度快，能够快速的处理摄像头传送过来的信息。低功耗特性能够有效节省能源，而工业级温度范围则确保其在各种恶劣环境下稳定工作。片上资源丰富，具有很多外围接口，可拓展性强，灵活性高。完全可以实现本系统的各个设计任务，具有良好的响应速度。瑞芯微开发板实物图如图 2-5 所示。

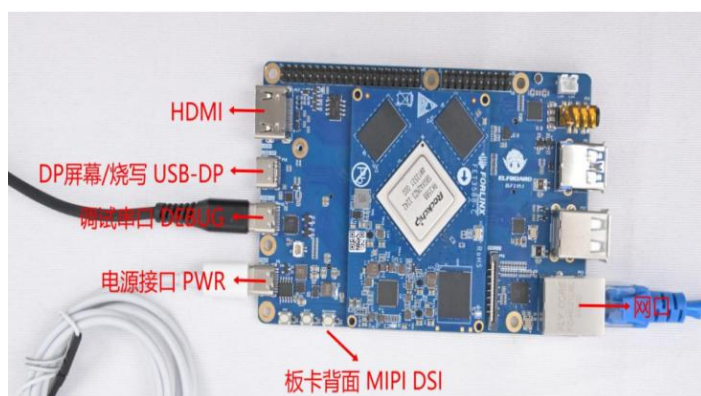


图 2-5 瑞芯微 RK3588 开发板

2.2.1.2 光电感知模块

系统采用高清工业级摄像头作为核心视觉感知设备，具备 1080P 及以上分辨率，配合广角镜头设计，水平视场角 98° ，竖直射场角 57° ，可实现对大范围水域的有效覆盖。摄像头具有 WDR（宽动态范围）和 3D 降噪功能，在强逆光、水面反光、夜间补光等复杂光照条件下仍能保持较高图像质量。通过 MIPI-CSI

接口与主控平台直连，数据传输带宽充足，避免了 USB 摄像头的编解码延时。同时配备环形麦克风阵列，覆盖全向拾音范围，用于采集现场环境音频以进行呼救声识别。摄像头视觉覆盖范围如图 2-6 所示。



图 2-6 高清广角无畸变摄像头视野范围实拍图

2.2.1.3 动力系统

在动力系统设计中，针对传统有刷电机存在的机械磨损大、寿命短、效率低及易产生电火花等问题，本系统选用双无刷直流电机（BLDC）作为推进核心。无刷电机采用电子换向方式替代机械换向，具有运行平稳（噪声低于 50dB）、效率高（>85%）、免维护等优点，同时避免了碳刷磨损带来的性能衰减和电气安全隐患。电机密封等级达 IP68，适用于长期浸水环境。在控制层面，通过 30A 电子调速器（ESC）实现 PWM 精细调速，配合 PID 闭环算法可达到毫秒级响应，最大推力 80N，支持设备以最高 1.5m/s 的速度航行。无刷电机模块实物如图 2-7 所示。



图 2-7 无刷电机模块实物图

2.2.1.4 压力传感器

系统引入薄膜压力传感器用于感知救生圈与落水人员之间的接触状态。传感器安装于救生圈流体型开口内侧及握把区域，量程 0-60N，响应时间小于 10 毫秒。当落水人员接触或抓靠救生圈时，传感器实时检测到压力变化并通过 GPIO 中断信号反馈至主控系统，触发报警抑制和救援成功状态上报。该机制使系统能够准确判断救援是否成功执行，辅助管理人员及时掌握现场情况。压力传感器如图 2-8 所示。



图 2-8 压力传感器实物图

2.2.1.5 通信模块

系统通过主控平台内置的 Wi-Fi6 (AX200) 模块实现设备端与云平台的无线数据交互。Wi-Fi6 理论传输速率可达 433Mbps，在覆盖良好的水域场景中能够稳定承载视频流、识别结果及设备状态数据的实时上传需求。同时板载蓝牙 4.2，可用于近距离设备配置与维护。

为扩展野外部署场景的适用性，系统预留了 4G/5G 蜂窝通信模块的扩展接口，可在无 Wi-Fi 覆盖的开放水域切换至移动网络进行数据传输。通信模块同时可集成卫星定位功能（GPS/北斗双模），为自主导航提供位置信息。AX200 网卡实物图如图 2-9 所示。



图 2-9 AX200 网卡实物图

2.2.2 机械结构介绍

2.2.2.1 系统总体结构图

基于 ANSYS 流体力学仿真对救生圈船体外形进行了系统性优化设计。通过 AutoCAD 和 SolidWorks 完成三维建模，采用 3D 打印技术制作原型机外壳。流体型结构参数：曲率半径 $R=120\text{mm}$ ，开口宽度约 400mm ，总长度约 700mm ，适配标准成人救生圈尺寸。内部腔体集成主控板、电池、电机及传感器模块，外部采用防水密封设计（IP67 等级）。原型机外观如图 2-10 所示。



图 2-10 救生圈外观 3D 建模图

2.2.2.2 多船舱结构设计

为了提升救生圈的防水性，我们借鉴了目前世界上通用的多船舱结构设计，保证在长时间使用过程中，即使系统出现局部破损，也能保证有充分的浮力可以满足救援需要。多船舱结构如图所示。同时，为了保障电路的稳定运行和系统结构的紧密性，我们将开发板和各种拓展电路通过立柱支撑在救生圈中间，保证系统稳定运行。多船舱结构 3D 建模图如图 2-11 所示。

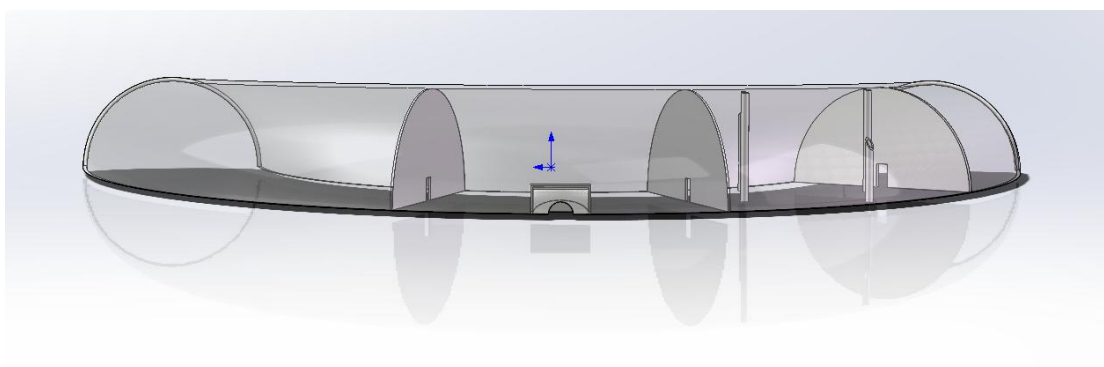


图 2-11 多船舱结构 3D 建模图

2.2.2.3 双层防缠绕设计

在我们实际测试中发现，水域中会出现一些杂物，有被缠绕进电机的风险，所以我们设计了双层防缠绕结构，极大的避免了这一情况。也保证了在救援过程中溺水者的衣物以及手指等被电机缠绕，避免二次伤害。双层防缠绕设计结构如图 2-12 所示。

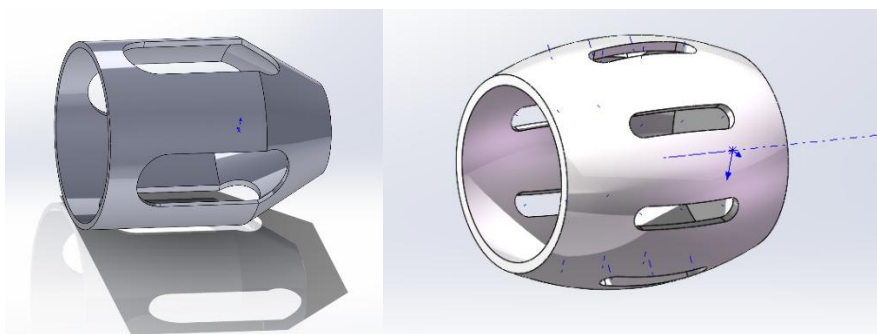


图 2-12 双层防缠绕结构图

2.2.2.4 岸基固定支架设计

本系统在岸基固定释放支架上设计了声光报警与舵机释放联动机制，使系统在识别到溺水风险后，能够在水域现场同步完成警示和救生圈释放。

当云端服务器接收到有效溺水报警后，向执行端控制节点下发释放指令。执行端收到指令后，首先触发支架上的 LED 警示灯和蜂鸣器，使现场能够通过灯光频闪和声音报警及时感知异常情况；同时，控制节点驱动舵机由锁定角度转动至释放角度，解除救生圈固定卡扣，使智能救生圈脱离岸基支架并进入水中开展后续救援。岸基固定支架建模效果如图 2-13 所示，声光组件如图 2-14 所示。



图 2-13 舵机装配体建模效果图



图 2-14 声光组件

2.2.3 电路各模块介绍

2.2.3.1 主控外接电路

主控平台的 MIPI-CSI、UART、GPIO、PWM 等接口按照功能需求分配，分别连接摄像头、通信模块、电机电调、压力传感器等外设。核心板引脚电路原理图如图 2-15 所示。

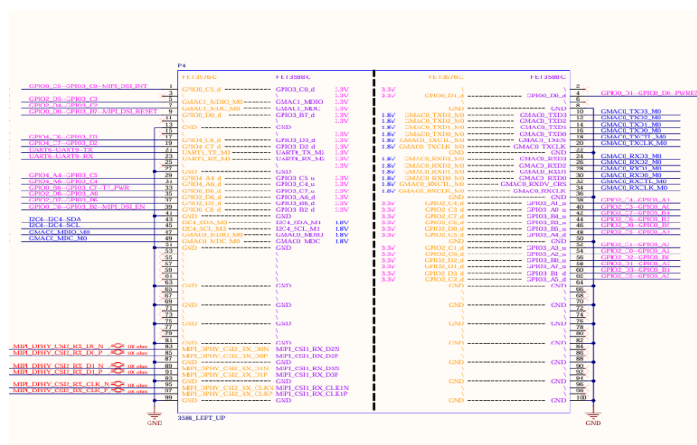


图 2-15 核心板引脚电路原理图

2.2.3.2 传感器集成电路

为提高系统集成度，将压力传感器、麦克风阵列等各种模块化接口集成于同一块 PCB 上，采用 I2C 总线和 GPIO 中断与主控通信，简化装配并提高系统的集成度和可靠性。传感器集成模块 PCB 如图 2-16 所示。

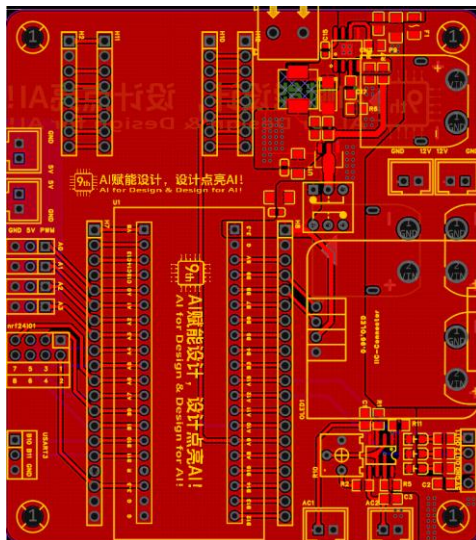


图 2-16 传感器集成模块 PCB 设计图

2.2.3.3 辅助电源电路

系统采用 3S/8Ah 锂聚合物（LTPO）动力电池供电。由于各模块对电源电压需求各异，设计了 $12V \rightarrow 5V \rightarrow 3.3V$ 三级降压辅助电源模块，分别采用开关电源 DC-DC 稳压器和 LDO 方案满足不同负载需求。辅助电源模块原理图如图 2-17 所示。

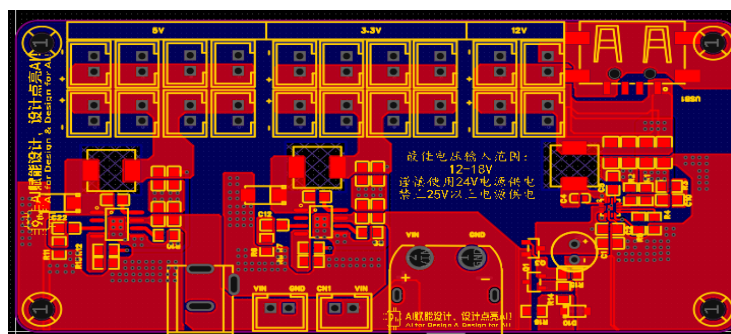


图 2-17 辅助电源模块 PCB 设计图

2.2.3.4 反激电源电路

在使用多级降压辅助电源模块供电的同时，为防止无刷电机工作时产生的大

电流引起地平面抬升从而干扰主控板正常运行，特意设计了反激隔离电源模块，通过变压器实现电机驱动电路与主控数字电路地平面之间的电气隔离，保障主控在电机启动的时候也能稳定运行。反激电源模块 PCB 及实物如图 2-18 所示。

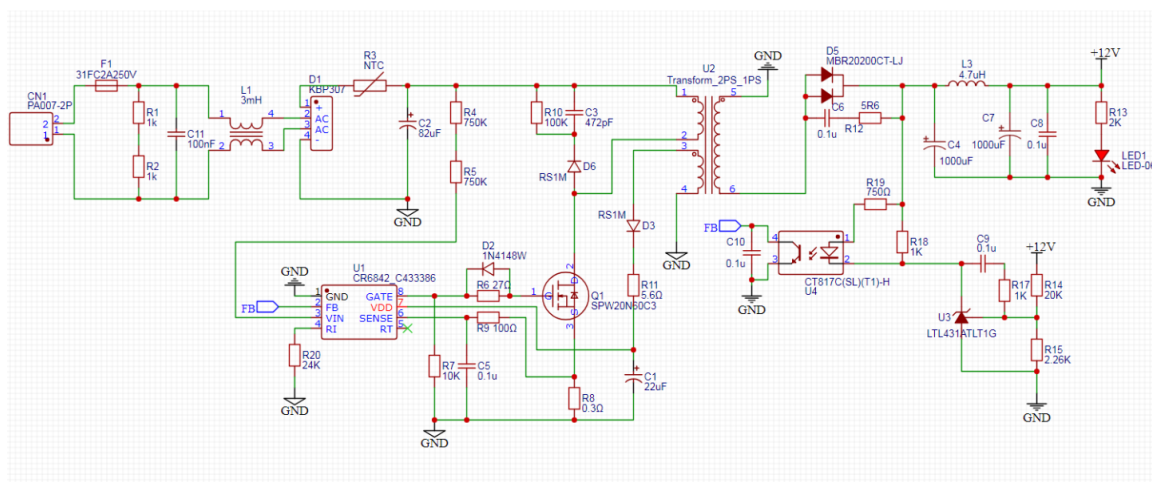


图 2-18 反激电源模块原理图

2.2.3.5 太阳能充电电路

此外，为了保证我们智能舵机支持户外全天候 24 小时工作，我们为其添加了太阳能充电电路，当阳光照射使太阳能板输出电压 V_{SUN} 大于等于电池电压 V_{bus} 时，整套电路由太阳能板供电，同时通过如韵电子的 CN3722 太阳能充电管理电路实现最大功率点跟踪（MPPT），为舵机 2S 电池充电；太阳能板输出电压 V_{SUN} 小于 V_{BUS} 时，电路重新回归电池供电，此时 CN3722 自动进入低功耗的睡眠模式。太阳能充电电路原理图如图 2-19 所示

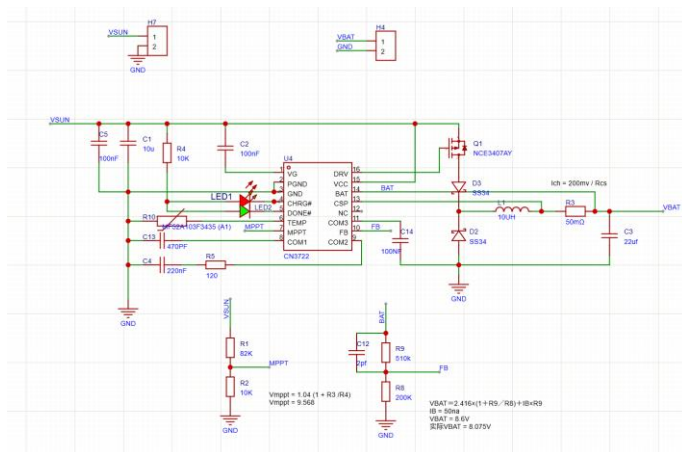


图 2-19 CN3722 太阳能充电电路原理图

2.3 软件系统介绍

2.3.1 软件整体介绍；

本系统软件采用"边缘推理+云端决策+多端协同"的分层架构，部署于边缘计算终端、云服务器、舵机控制节点、Web 管理端和移动端五类设备上。软件整体流程如图 2.20 所示。

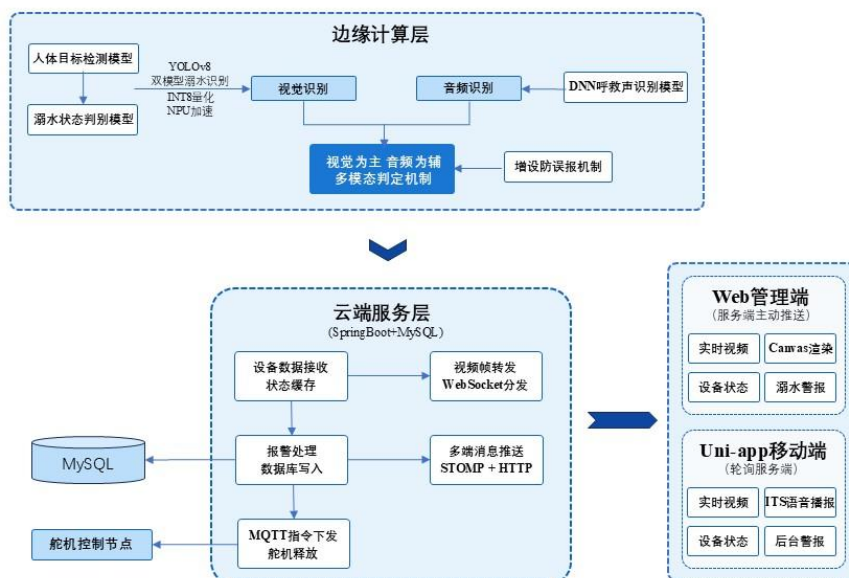


图 2-20 软件系统总体流程图

边缘计算终端运行基于 NPU 推理框架的 YOLOv8 双模型协同识别软件，负责实时视频采集、AI 推理、多源感知融合、选择性数据上传和视频帧推送。云端服务平台基于 SpringBoot 框架开发，集成嵌入式 MQTT 消息代理，负责设备数据接收与缓存、报警联动决策、数据库持久化、视频帧转发和多端消息推送。舵机控制节点运行 MQTT 客户端固件，接收释放指令并驱动舵机完成救生圈投放。Web 管理端采用 Vue3 框架，通过 WebSocket 双通道分别接收视频帧和状态数据，实现实时监控与报警管理。移动端基于 Uni-App 开发，通过 HTTP 轮询获取设备状态，集成 Android 原生 TTS 引擎实现语音报警。

软件系统通过 HTTP、WebSocket、STOMP 和 MQTT 四种通信协议实现模块间数据交互，形成从"AI 感知→云端决策→自动救援→多端反馈"的完整软件闭环。

2.3.2 软件各模块介绍

2.3.2.1 边缘计算终端软件

边缘计算终端软件以 Python 脚本为主控框架，通过子进程方式启动 C++ 编写的 NPU 推理引擎，两者通过标准输出管道进行数据交换。整体流程为"量化模型加载→NPU 推理→结果后处理→防误报确认→选择性上传→视频帧推送"，各环节均采用异步非阻塞设计。

(1) 量化模型加载与 NPU 推理模块

标注框 JPEG 图像帧（写入内存盘/dev/shm/frames_int8_pixel_box）

模块启动时加载 INT8 量化后的溺水识别模型和人员检测模型，初始化 NPU 推理引擎并分配输入输出张量。推理循环中，摄像头帧经尺寸缩放和通道转换后构建输入张量，调用 NPU 完成前向推理。双模型采用协同策略：人员检测模型以较低频率运行定位水面人员，溺水模型每帧运行判断目标异常状态。推理输出经置信度阈值筛选、坐标还原和非极大值抑制处理后，生成结构化检测文本和带标注框的 JPEG 图像。边缘端双模型推理流程图如图 2-21 所示。

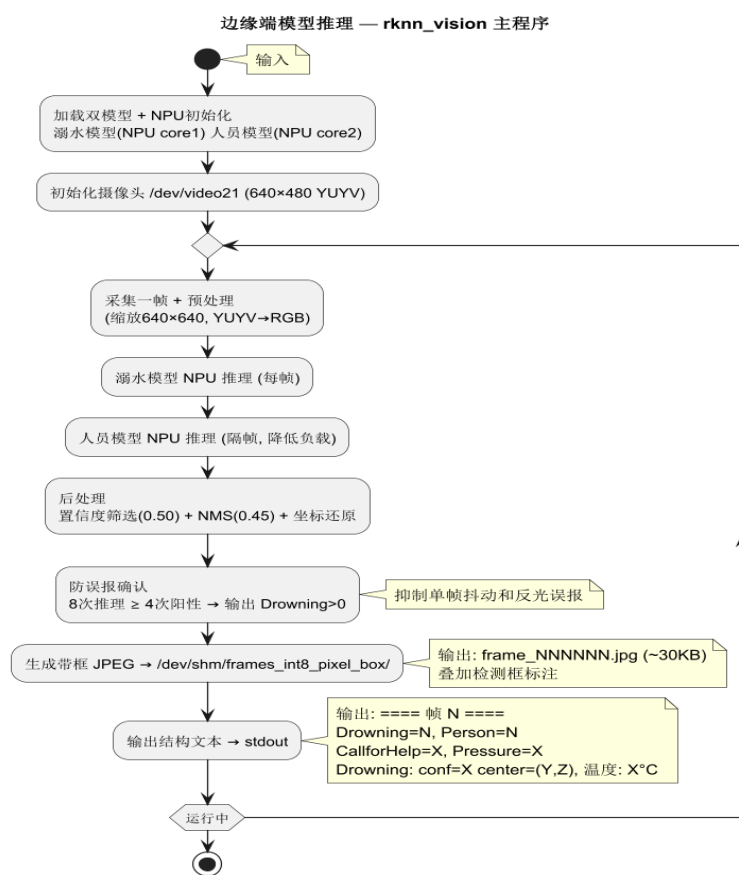


图 2-21 软件系统总体流程图

(2)数据解析与选择性上传模块

Python 主控脚本通过正则表达式实时解析模型输出的文本行，提取帧号、溺水人数、岸上人数、呼救声标志、压力值、温度和目标坐标。采用选择性上传策略：呼救声变化、压力状态变化时立即上传；连续多帧确认溺水后上传报警；温度按 5 秒周期定时上传。所有上传操作提交至单线程线程池异步执行，不阻塞主解析循环。连续帧确认窗口为多帧中阳性帧数超过阈值方判定为有效报警，抑制单帧模型抖动导致的误报。

(3)视频帧推送模块

视频推送模块独立于数据上传模块运行。以 8 毫秒为周期轮询内存盘目录，通过正则匹配帧号定位最新帧文件。采用容量为 1 的帧队列传递路径，入队前清空旧数据，保证推送的始终是最新帧。读取 JPEG 字节后按自定义二进制协议打

包发送至云端 WebSocket。连接断开自动重连，启动时清空内存盘历史帧，每 2 秒清理旧帧文件控制在 5 张以内。

2.3.2.2 云端服务平台软件

云端服务平台采用 SpringBoot3.x+MyBatis+MySQL 技术栈，内嵌 MoquetteMQTTBroker 和 EclipsePahoMQTT 客户端，集成 SpringWebSocket 实现 STOMP 消息推送。软件按功能划分为设备数据接收、报警处理与 MQTT 下发、视频帧转发和多端消息推送四个核心模块。

(1)设备数据接收与缓存模块

DeviceController 接收边缘端上传的表单数据，解析各字段后构建 DeviceStatus 对象。DeviceService 执行溺水边沿检测（计数器从 0 变为正数时触发写库），经防重复检查（数据库已有 PENDING 记录则不再插入）后写入 alarm_record 表。设备状态同时更新至内存缓存并通过 STOMP 推送至 Web 端。压力传感器触发时自动将 PENDING 记录改为 COMPLETED 状态，报警抑制逻辑使综合报警值归零。

(2)报警处理与 MQTT 下发模块

ServoService 维护每个设备的溺水推送状态。首次接收到溺水帧时立即通过 PahoMQTT 客户端发布释放指令并推送 Web 端弹窗；持续溺水期间每隔 3 秒重复推送指令，推送前检查 MQTT 连接状态并自动重连。当数据库 PENDING 记录被人工确认或压力传感器自动确认后，推送状态归零，停止下发。

(3)视频帧转发模块

FrameWebSocketHandler 接收边缘端上传的二进制帧，解析设备 ID 和 JPEG 数据后完成三项操作：将帧字节存入 FrameService 内存缓存供快照接口使用；通过 BrowserFrameHandler 向所有订阅该设备的浏览器 WebSocket 会话广播帧数据；从设备缓存读取温度后构造帧元数据推送至 STOMP 主题。广播采用 100 毫秒超时保护，防止单个慢客户端阻塞整条推流管道。

(4)RESTAPI 模块

提供以下接口供 Web 端和移动端调用：GET/device/latest（设备状态查询，

含 DB 报警锁存)、GET/device/snapshot/{id} (实时快照)、GET/api/alarm/list (报警记录列表)、POST/api/alarm/{id}/acknowledge (确认报警完成)、POST/api/servo/trigger (手动触发舵机)、POST/api/servo/reset (舵机状态复位)、POST/user/login (JWT 认证)。

2.3.2.3 舵机控制节点

控制节点启动后依次执行 WiFi 连接、MQTTBroker 连接和主题订阅。收到消息后通过 Arduino Json 库解析 JSON，校验 cmd 为 RELEASE 且 deviceId 匹配后，通过 Servo 库输出 PWM 驱动舵机。释放后启动 2 秒定时器自动复位，物理复位按钮支持手动操作。集成硬件看门狗定时器和自动重连机制，断线后 5 秒间隔重试。

2.3.2.4 Web 管理端软件

Web 端采用 Vue3+Vite+ElementPlus 技术栈，按 api/stores/composables 三层分离架构组织。Detection View 页面左侧为 Canvas 实时视频区域，右侧为状态面板。视频帧通过独立 WebSocket 接收 JPEG 二进制数据，使用 createImageBitmap 解码后以 requestAnimationFrame 调度绘制到 Canvas，避免重复设置画布尺寸带来的性能开销。设备状态和报警事件通过 STOMPoverWebSocket 接收，报警弹窗采用 CSS 玻璃态呼吸灯动画。报警列表每 5 秒自动刷新，PENDING 状态的报警可点击“确认完成”按钮解除，确认后立即推送更新至 STOMP 使前端报警归零。

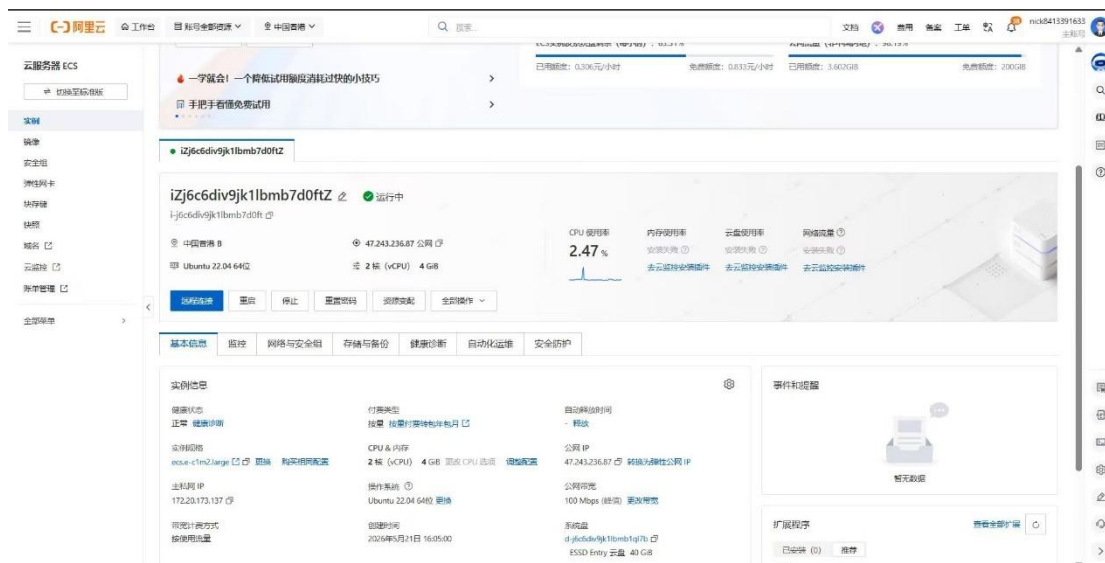


图 2-22 阿里云服务器示意图

2.3.2.5 移动端软件

移动端基于 Uni-App 框架开发，主页展示设备选择器、状态栏、呼救报警和监控报警模块、设备信息卡片。副页面为全屏监控画面，定时拉取服务器快照展示。报警触发时调用 Android 原生 TextToSpeechAPI 进行中文语音播报（语速 0.65），同时通过 NotificationManager 发送系统通知栏消息。语音播报设有 8 秒冷却机制防止叠加，通知栏消息支持点击返回 App。设备状态以 400 毫秒周期轮询，报警状态经边缘检测后触发播报和推送。移动端 app 数据处理流程如图所示。

第三部分 完成情况及性能参数

3.1 整体介绍

本作品以瑞芯微 RK3588 芯片为核心，其 6TOPS 超强算力和多核异构架构，保障复杂场景下实时数据处理；采用 AI 算法实现落水者的精准识别，优化后的 YOLOv8 算法每秒 30 帧解析水域画面，通过高清广角摄像头实时捕捉水面图像，为了提高落水者识别的实时性，该模块对 YOLOv8 模型进行精细剪枝量化，并利用大量的多场景数据集提升识别率；设计救援最优导航算法，基于视觉定位与 IMU 姿态反馈数据，自主规划最短路径，结合无刷电机与水下推进器实现救生圈

的快速自主救援；使用改进的流体型外观，相较于传统的救生圈，该外观在救生时受到的阻力更小，极大的提升了救援的效率；同时开发上位机软件，配合救生圈的 WiFi 6/4G/5G 通信模块，同步实时数据至管理平台，支持水域管理员远程监控设备状态与报警信息。原型机外观如图 3-1、3-2 所示。



图 3-1 救生圈外观原型机展示（斜 45°、正面图）



图 3-2 救生圈实地部署效果图

3.2 工程成果

3.2.1 机械成果

我们通过 SolidWorks、AutoCAD 对救生圈螺旋桨外壳进行 3D 建模设计并打

印，外壳模型图如图 3-3 所示。



图 3-3 3D 打印外壳

3.2.2 电路成果

(1) 传感器集成电路模块。



图 3-4 传感器集成模块实物图

(2) 辅助电源模块。



图 3-5 辅助电源模块实物图

(3) 舵机控制模块。

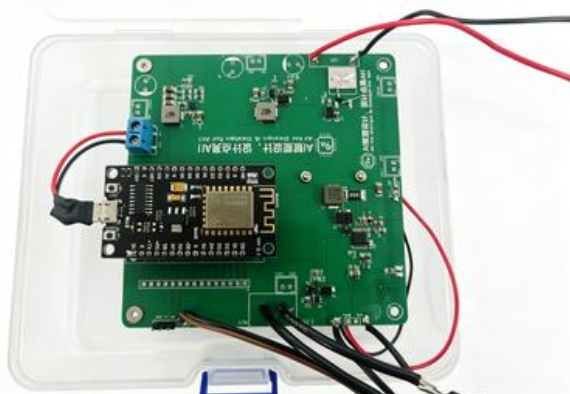


图 3-6 舵机控制电路实物图

(3) 反激隔离电源模块。

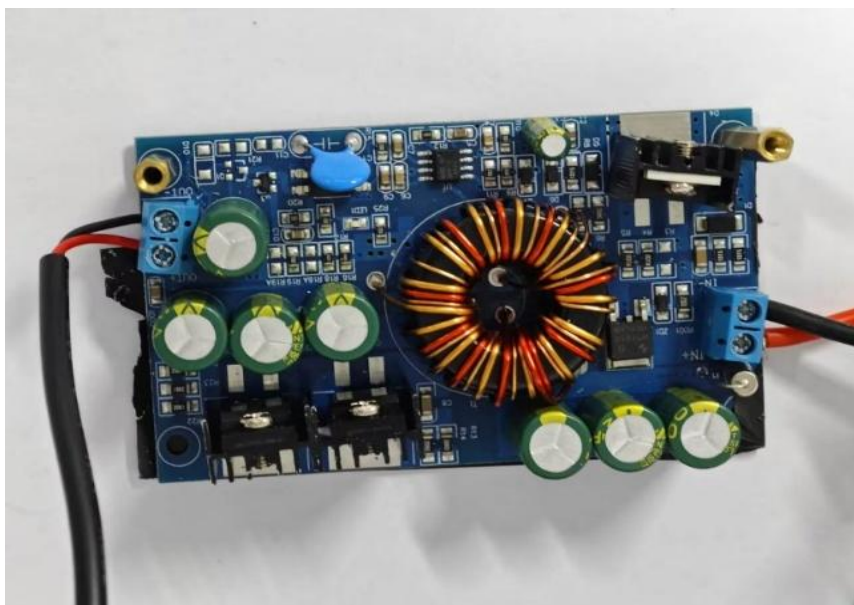


图 3-7 反激电源模块实物图

3.2 软件成果

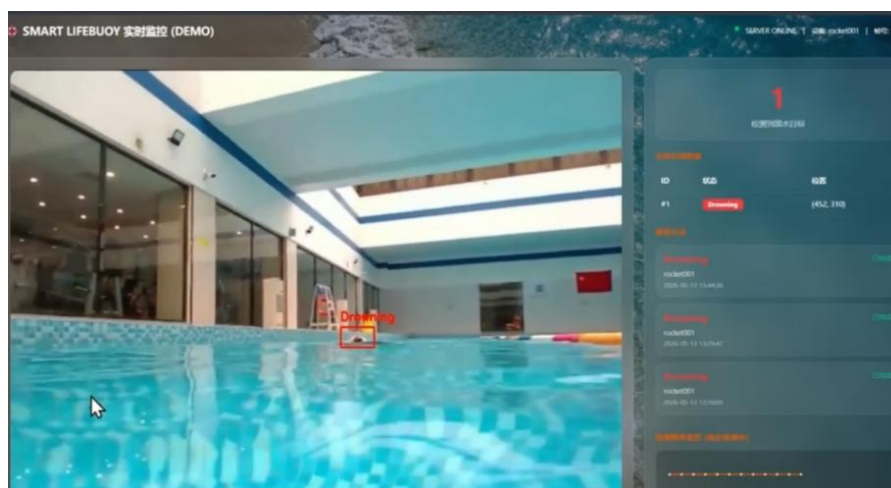


图 3-8 web 端管理页面图

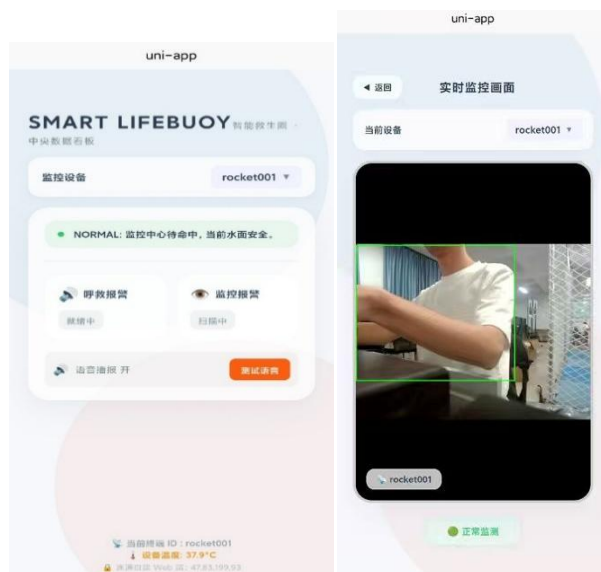


图 3-9 移动端界面截图

3.3 特性成果

3.3.1 基于 YOLOv8 双模型及 DNN 音频识别的多模态溺水识别算法

系统采用多源感知融合策略降低误报率，并通过 INT8 量化和 NPU 提升识别速率。视觉部分通过 130°广角摄像头采集水域画面，利用 YOLOv8 双模型先完成人体目标检测，再对目标状态进行溺水风险判别；音频部分通过麦克风采集现场声音，利用 DNN 呼救声识别模型对有效呼救声进行辅助检测。边缘端以视觉识别结果为主要依据，并结合连续帧检测结果和呼救声识别结果进行综合判断，避免单帧误检、水面反光、漂浮物干扰或偶发噪声造成误报警，从而提高险情判断的稳定性和可靠性。

3.3.1.1 计算机视觉

视觉识别部分采用 YOLOv8 双模型协同策略，按照“人体目标检测—溺水状态判别”的流程进行处理。系统首先通过人体检测模型定位水面人员目标，再利用溺水状态识别模型对目标姿态和行为状态进行风险判断。相比单一模型直接判断溺水状态的方式，该策略能够减少水面反光、漂浮物、岸边行人等复杂背景带来的干扰，提高目标定位和险情判断的稳定性。

为满足边缘端实时识别需求，系统将视觉模型转换为适配 RK3588 开发套件的推理格式，并采用 INT8 量化方式降低模型计算量和存储占用，使模型能够充分利用板载 NPU 进行加速推理。同时，边缘端引入连续多帧确认机制，对短时间内的检测结果进行稳定判断，避免单帧模型抖动、偶发遮挡或单帧误检直接触发报警。视觉分析效果如图 3-10 所示。

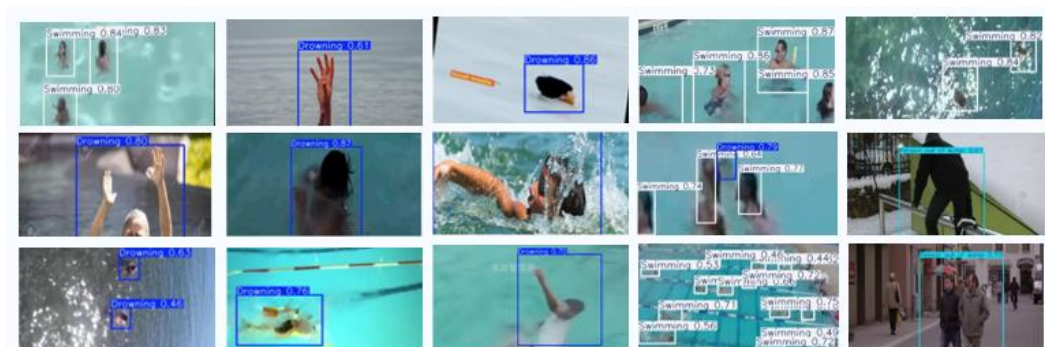


图 3-10 YOLO_V8 模型部分识别效果图

3.3.1.2 呼救声识别

音频识别部分用于对视觉检测进行辅助判断。系统通过麦克风阵列采集水域周边环境声音，并对原始音频进行预处理，包括分帧、降噪和归一化等操作，以降低背景噪声对识别结果的影响。随后，系统提取音频的梅尔频谱特征，将声音信号转换为适合模型处理的特征表示，并输入 DNN 呼救声识别模型进行判断，输出呼救声概率结果。

当模型检测到有效呼救声时，系统不会直接替代视觉识别结果，而是将其作为辅助报警依据，与视觉端的溺水识别结果共同参与多模态融合决策。该机制能够在目标被遮挡、距离较远或视觉识别不稳定的情况下提高系统对险情的感知能力；同时也可避免单一声音噪声直接触发报警，提高整体判断的可靠性。呼救声识别流程如图 3-11 所示。



图 3-11 呼救处理流程图

3.3.2 基于双 PID 算法的精准姿态控制

本项目使用视觉定位来计算溺水人员与视觉中心的差值，通过 x 轴差值来计

算两侧电机之间的差速，通过差速来调整救生圈的行进方向；通过 y 轴差值来计算电机运行的速度，y 轴数值越大，救生圈离溺水人员的距离越远，电机的速度越大，依据该算法可以实现救生圈在救援过程中的智能减速，在靠近溺水人员时候自动停止，避免对溺水人员的二次伤害。

PID 控制算法通过比较姿态采集模块返回的数据与期望姿态，对检测到的误差进行比例、积分和微分处理，如图 3-10。然后将这些调整值应用于电机，以修正姿态。比例调节提供快速响应和显著调整效果，但可能导致稳定性不足和过度调节。积分调节能够消除长期误差和外部干扰，但可能引起系统震荡。微分调节则通过预测设备未来状态并进行相应调整，同时抑制比例调节的过度反应，增强系统稳定性。综合运用 PID 控制，系统能够全面考虑过去、当前和未来状态，实现稳定运行。

三种调节对应的时间特性如表 3-1。简单来说，模拟 PID 控制器中各环节的作用如下：

表 3-1 PID 三种调节对应的时间意义

调节类型	意义	时间关系
比例调节 P	反映当前值与当前误差的差值	当前与当前
积分调节 I	当前误差是在此之前的误差的累计	当前与之前
微分调节 D	用近期的误差来预测未来误差的走势	当前与未来

（1）比例环节

及时成比例地反应控制系统的偏差信号 $e(t)$ ，偏差一旦产生，控制器立即产生调节作用,以减少偏差。

（2）积分环节

主要用于消除静差来提高系统的误差度。积分作用的强弱取决于积分时间常数 T 。 T 越大，积分作用越弱，反之越强。

（3）微分环节

能够反应偏差信号的变化趋势，即偏差信号的变化率。并能在偏差信号变得

太大前，在系统中引入一个有效的早期修正信号，从而加快系统的动作速度，减少调节时间。通常，单片机控制为离散的采样控制，因此，所使用的是数字 PID 控制。通过将模拟表达式中的积分、微分运算用数值计算方法来逼近，便可实现数字 PID 控制。

第四部分 总结

4.1 可扩展之处

4.1.1 功能维度扩展

可预留接口接入生命体征监测模块，救援时同步采集落水者心率、体温等数据，为后续医疗救治提供参考；扩展水质、水文监测模块，将系统从单一救援延伸至智慧水域综合管理；增加红外热成像、声呐探测模块，提升夜间、浑浊水域的感知能力。

4.1.2 场景适配扩展

针对水库、急流河道优化推进系统与地形适配能力，拓展至渔船码头、远海等场景；将核心技术复用至冰面救援、城市内涝救援等其他应急场景，拓宽产品应用边界。

4.1.3 智能协同扩展

引入多设备组网协同技术，实现多台救生圈集群作业，应对大面积水域、多人遇险场景；引入强化学习算法优化动态路径规划，结合实时水流风力数据自适应调整航线，进一步提升救援效率。

4.1.4 能源续航扩展

增加无线充电等补能方式，延长设备续航；设计智能休眠唤醒机制，无风险时段低功耗待机，平衡续航与响应速度。

4.2 心得体会

本次“基于 RK3588 的多模态分析与自主搜救水上救生系统”项目开发，是一次跨学科、全链路的工程实践，从最初的需求构思到最终的原型落地，团队在技术攻坚与协作配合中都收获了长足的成长。

在方案选型阶段，团队围绕算力平台、通信方案、控制算法展开了多轮论证。算力层面，对比了传统单片机、低端嵌入式平台与 RK3588 高端边缘平台的差异，最终选择 RK3588 正是看中其充足的 AI 算力与丰富接口，能够支撑多模态识别与边缘端实时处理的需求，事实也证明这一选择为系统的性能上限提供了充足保障。通信方案上，对比了 WiFi、LoRa 与 4G/5G 方案，考虑到开放水域的广覆盖需求与视频传输的带宽需求，最终选定 4G/5G 为主、WiFi 为辅的方案，兼顾了部署灵活性与传输性能。

在技术攻坚阶段，模型轻量化与边缘部署是首个难点。原始 YOLOv8 模型体积大、算力要求高，直接部署无法达到实时性要求。团队逐步尝试结构剪枝、INT8 量化等手段，反复调整剪枝比例与量化参数，在精度与速度间寻找平衡点；同时适配 RKNN 工具链，解决了模型转换、层兼容等一系列问题，最终成功将模型压缩至 89MB 并实现 30FPS 推理。运动控制调试是另一项挑战，PID 参数整定需要反复测试，从最初的航向振荡、转向过度，到逐步调整比例、积分、微分系数，再加入水流补偿与姿态融合，最终实现了平稳精准的航行控制，这个过程让我们深刻体会到控制理论与工程实践结合的重要性。

团队协作方面，项目涵盖硬件、算法、软件、结构多个方向，成员根据专长明确分工，同时保持高频沟通。硬件调试时算法同步优化模型，结构建模时软件同步开发平台，各模块并行推进，再通过接口联调整合为完整系统。遇到跨模块问题时，大家共同定位原因、协商解决方案，在协作中既发挥了个人优势，也培养了系统工程思维。

整个项目过程中，我们也深刻感受到科技创新服务社会的价值。水上安全关系到无数人的生命安全，传统救援模式的局限让很多悲剧本可避免。当亲手打造的设备成功识别险情、自主驶向目标的那一刻，所有调试的辛苦都有了切实的意义。这也让我们更加坚定了用专业知识解决实际问题的方向，未来将继续打磨产品、优化性能，让技术真正守护生命安全。

第五部分 参考文献

- [1]李明, 赵宇, 孙晓。基于 YOLOv8 的水域目标检测算法优化及应用研究[J].智能系统学报, 2024,19(03):456-468.
- [2]王强, 刘娜, 陈辉。智能水域救援设备的技术演进与应用趋势[J].应急技术与管理, 2023,8(02):32-40.
- [3]张峰, 吴敏, 郑涛。多传感器融合的水域环境监测系统设计与实现[J].测控技术, 2022,41(11):78-83.
- [4]赵亮, 孙丽, 周凯。水上救援设备能源补给与低功耗设计研究[J].电源技术应用, 2023,16(04):56-63.
- [5]陈宇, 王浩, 李阳。水域救援设备组网协同技术研究[J].通信技术, 2024,57(01):123-130.
- [6]刘杰, 张慧, 王鹏。复杂水域环境下智能救援设备的场景适配研究[J].水利信息化, 2023,15(03):28-35.
- [7]吴芳, 郑明, 崔浩。水域救援设备人机协同及生命体征监测系统设计与实现[J].生物医学工程学报, 2024,41(02):312-320.
- [8]王宇, 李娟, 张平。基于流体力学仿真的救生设备结构优化设计[J].船舶工程, 2022,44(09):67-73.
- [9]GlennJocher,AyushChaurasia,JingQiu.UltralyticsYOLOv8[EB/OL].<https://github.com/ultralytics/ultralytics>,2024.
- [10]AndrewBanks,RahulGupta.MQTTVersion3.1.1OASISStandard[S].OASISOpen,2014.

第六部分附录

(1) 多模态识别代码核心片段

```
make > main.cpp

int main(int argc, char **argv)
{
    printf("===== dual_model_camera_int8_multi VERSION DROWNING-80F4-UART-DEBUG-FIX =====%s", "\n");

    if (argc < 3) {
        printf("用法:\n");
        printf(" %s ./best_int8_fixed.rknn ./yolov8m_int8_fixed.rknn\n", argv[0]);
        return -1;
    }

    const char *drowning_model_path = argv[1];
    const char *person_model_path = argv[2];

    printf("[MODEL PATH] Drowning: %s\n", drowning_model_path);
    printf("[MODEL PATH] Person: %s\n", person_model_path);
    printf("[THRESH] Drowning=%.2f Person=%.2f NMS=%.2f\n",
        DROWNING_CONF_THRESH, PERSON_CONF_THRESH, NMS_THRESH);
    printf("[CONFIRM] Drowning recent %d model-runs >= %d positive runs will output/alarm\n",
        DROWNING_CONFIRM_WINDOW, DROWNING_CONFIRM_MIN);
    printf("[THERMAL] LOW=%d MID=%d HIGH=%d CRITICAL=%d, temp log every %d ms\n",
        TEMP_LOW, TEMP_MID, TEMP_HIGH, TEMP_CRITICAL, TEMP_LOG_INTERVAL_MS);
    printf("[UART STOP] no confirmed Drowning for %d ms -> send target_id=0 x=0 y=0 w=0 h=0, repeat every %d ms\n",
        DROWNING_STOP_TIMEOUT_MS, DROWNING_STOP_REPEAT_MS);

    // 初始化串口
    UartSender uart(UART_DEVICE, UART_BAUD);
    if (!uart.isReady()) {
        printf("警告: 串口 %s 初始化失败, 将继续运行但无法通信\n", UART_DEVICE);
    } else {
        printf("串口 %s 初始化成功, 波特率 %d\n", UART_DEVICE, UART_BAUD);
    }

    gpio_init(TARGET_GPIO);
    help_call_init();

    if (camera_init() < 0) {
        return -1;
    }
}
```

(2) PID 控制代码核心片段



```
134 // }
135 if (!left_idle_enable && vision_servo_enable)
136 {
137     /* 1. 无效目标: x=0且y=0 → 直接停车 */
138     if (target.x == 0 && target.y == 0)
139     {
140         Motor_SetSpeed(0, 0);
141         OLED_ShowSignedNum(5, 4, 0, 3);
142         OLED_ShowSignedNum(5, 10, 0, 3);
143         OLED_ShowString(6, 7, "0 ");
144         continue;
145     }
146
147     /* 计算目标尺寸比例, 用于近距减速 (保留原安全逻辑) */
148     float width_ratio = (float)target.width / CAMERA_WIDTH;
149     float height_ratio = (float)target.height / CAMERA_HEIGHT;
150     float size_ratio = (width_ratio > height_ratio) ? width_ratio : height_ratio;
151
152     float speed_scale = 1.0f;
153     if (size_ratio >= MAX_RATIO)
154     {
155         speed_scale = 0.0f;
156     }
157     else if (size_ratio > MIN_RATIO)
158     {
159         speed_scale = 1.0f - (size_ratio - MIN_RATIO) / (MAX_RATIO - MIN_RATIO);
160     }
161
162     /* 2. 目标在视野中央 → 双电机全速前进, 方向为0 */
163     if (abs((int)(target.x - CENTER_X)) <= PID_DEAD_ZONE_X
164         && abs((int)(target.y - CENTER_Y)) <= PID_DEAD_ZONE_Y)
165     {
166         int16_t full_speed = (int16_t)(100 * speed_scale);
167         Motor_SetSpeed(full_speed, 0);
168
169         OLED_ShowSignedNum(6, 7, (int16_t)(speed_scale * 100), 3);
170         OLED_ShowSignedNum(5, 4, 0, 3);
171         OLED_ShowSignedNum(5, 10, full_speed, 3);
172     }
173
174     /* 3. 目标不在中央 → X方向PID修正航向, 基础速度适当降低 */
175     else
176     {
177         // X方向PID计算转向修正量
178         float out_x = PID_Calc(&pid_x, (float)target.x);
179
180         // 非中央时基础速度为70%, 可根据实际手感调整
181         float base_speed = 70.0f;
182         // 根据X方向偏离程度再减速, 偏离越大速度越慢
183         float x_deviation = abs((float)(target.x - CENTER_X)) / (CAMERA_WIDTH / 2.0f);
184         float deviation_scale = 1.0f - x_deviation * 0.6f; // 最多减速60%
185
186         float final_speed = base_speed * deviation_scale * speed_scale;
187
188         Motor_SetSpeed((int8_t)final_speed, (int8_t)out_x);
189
190         OLED_ShowSignedNum(6, 7, (int16_t)(speed_scale * 100), 3);
191         OLED_ShowSignedNum(5, 4, (int16_t)out_x, 3);
192         OLED_ShowSignedNum(5, 10, (int16_t)final_speed, 3);
193     }
194 }
195
196 // 视觉断连检测 (超时5秒)
197 if (sysTick_ms - last_visual_tick > VISUAL_TIMEOUT_MS)
198 {
199     visual_online = 0;
200     OLED_ShowString(7, 7, "LOST");
201 }
```